

Name: Disha Dipak Naik Roll No: CS35 Batch : CS2

* Machine Learning Experiment Number 3

Aim : To implement a Multiple Linear Regression model from scratch using NumPy matrix operations to predict the selling price of used cars using multiple features from the CarDekho Used Car Dataset.

Learning Outcomes • Understand the working principle of Multiple Linear Regression.

- Perform data preprocessing and feature engineering.
- Convert categorical data into numerical features.
- Standardize numerical features.
- Implement the Ordinary Least Squares method using matrix operations.
- Train and test a regression model without using a ready-made regression function.
- Evaluate and interpret regression performance using RSS and R^2 .
- Understand the effect of individual features on the predicted price.

Dataset

Use the **CarDekho Used Car Dataset** (`car_data.csv`). [CarDekho Used Car Dataset](#)

Explain meaning of each attribute in this dataset in below comment section :

Here, **Selling_Price** is the target variable.

Software/Tools Required

- Python 3.x
- Google Colab / Jupyter Notebook
- NumPy
- Matplotlib

* Task 1: Set Up the Python Environment**

- Create a new Google Colab notebook.
- Import **NumPy** for numerical and matrix operations.
- Import **Matplotlib** for data visualization.

- Do not use ready-made machine-learning regression functions such as `LinearRegression()` from Scikit-learn.
- The regression model must be implemented using matrix operations.

```
# TASK 1: SET UP PYTHON ENVIRONMENT

# Import required libraries
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd

# -----
# Upload the dataset in Google Colab
# -----

from google.colab import files

uploaded = files.upload()

# Read the uploaded CSV file
df = pd.read_csv("cardekho_dataset.csv")

# -----
# Display the dataset
# -----

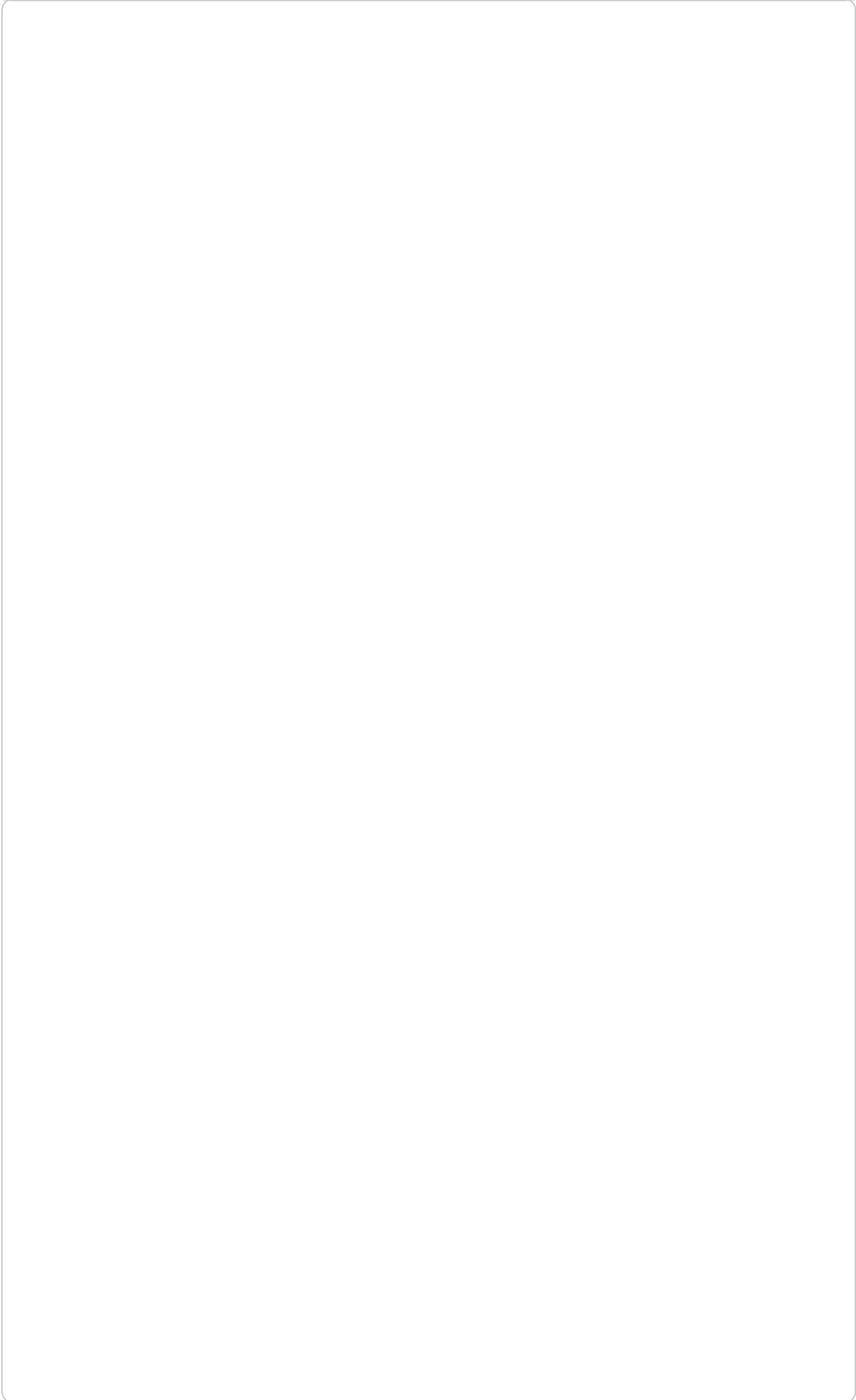
print("Dataset loaded successfully!")
print("\nFirst 5 rows of the dataset:")
display(df.head())

# Display number of rows and columns
print("\nDataset Shape:")
print(df.shape)

# Display column names
print("\nColumn Names:")
print(df.columns.tolist())

# Display information about the dataset
print("\nDataset Information:")
df.info()

# Display basic statistical information
print("\nStatistical Summary:")
display(df.describe())
```



Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving cardekho_dataset.csv to cardekho_dataset (1).csv

Dataset loaded successfully!

* Task 2: Load and Inspect the Dataset

- First 5 rows of the dataset:
- | Unnamed: 0 | car_name | brand | model | vehicle_age | km_driven | seller_type |
|------------|----------|---------|-------|-------------|-----------|-------------|
| 0 | Maruti | Maruti | Alto | 9 | 120000 | Individual |
| 1 | Hyundai | Hyundai | Grand | 5 | 20000 | Individual |
| 2 | Hyundai | Hyundai | i20 | 11 | 60000 | Individual |
| 3 | Alto | Maruti | Alto | 9 | 37000 | Individual |
- Download the `car_data.csv` file.
 - Upload the dataset to Google Colab.
 - Load the dataset into Python.
 - Display the first few records.
 - Determine the number of rows and columns.
 - Identify the numerical and categorical attributes.
 - Separate `Selling Price` as the target variable.

```
# TASK 2: LOAD AND INSPECT THE DATASET
# =====
Dataset Shape:
(15411, 14)
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Dataset Information:
# class pandas.core.frame.DataFrame
# RangeIndex: 15411 entries, 0 to 15410
# Data columns (total 14 columns):
#   Column Non-Null Count  Dtype
# 0  Unnamed: 0      15411 non-null    int64
# 1  car_name        15411 non-null    object
# 2  brand           15411 non-null    object
# 3  model           15411 non-null    object
# 4  vehicle_age     15411 non-null    int64
# 5  km_driven       15411 non-null    int64
# 6  seller_type     15411 non-null    object
# 7  fuel_type       15411 non-null    object
# 8  transmission_type 15411 non-null    object
# 9  mileage         15411 non-null    float64
# 10 engine        15411 non-null    int64
# 11 max_power     15411 non-null    float64
# 12 seats         15411 non-null    int64
# 13 selling_price 15411 non-null    int64
dtypes: float64(2), int64(6), object(6)
# memory usage: 1.6+ MB

# 3. Display first few records
# Statistical Summary:
#   Unnamed: 0  vehicle_age  km_driven  mileage  engine
# 0  0  9  120000  19.701151  1486.057751
# 1  1  5  20000  19.701151  1486.057751
# 2  2  11  60000  19.701151  1486.057751
# 3  3  9  37000  19.701151  1486.057751
# 4  4  6  30000  19.701151  1486.057751
# 5  5  6  30000  19.701151  1486.057751
# 6  6  6  30000  19.701151  1486.057751
# 7  7  6  30000  19.701151  1486.057751
# 8  8  6  30000  19.701151  1486.057751
# 9  9  6  30000  19.701151  1486.057751
# 10 10 6 30000 19.701151 1486.057751
# 11 11 6 30000 19.701151 1486.057751
# 12 12 6 30000 19.701151 1486.057751
# 13 13 6 30000 19.701151 1486.057751
# 14 14 6 30000 19.701151 1486.057751
# 15 15 6 30000 19.701151 1486.057751
# 16 16 6 30000 19.701151 1486.057751
# 17 17 6 30000 19.701151 1486.057751
# 18 18 6 30000 19.701151 1486.057751
# 19 19 6 30000 19.701151 1486.057751
# 20 20 6 30000 19.701151 1486.057751
# 21 21 6 30000 19.701151 1486.057751
# 22 22 6 30000 19.701151 1486.057751
# 23 23 6 30000 19.701151 1486.057751
# 24 24 6 30000 19.701151 1486.057751
# 25 25 6 30000 19.701151 1486.057751
# 26 26 6 30000 19.701151 1486.057751
# 27 27 6 30000 19.701151 1486.057751
# 28 28 6 30000 19.701151 1486.057751
# 29 29 6 30000 19.701151 1486.057751
# 30 30 6 30000 19.701151 1486.057751
# 31 31 6 30000 19.701151 1486.057751
# 32 32 6 30000 19.701151 1486.057751
# 33 33 6 30000 19.701151 1486.057751
# 34 34 6 30000 19.701151 1486.057751
# 35 35 6 30000 19.701151 1486.057751
# 36 36 6 30000 19.701151 1486.057751
# 37 37 6 30000 19.701151 1486.057751
# 38 38 6 30000 19.701151 1486.057751
# 39 39 6 30000 19.701151 1486.057751
# 40 40 6 30000 19.701151 1486.057751
# 41 41 6 30000 19.701151 1486.057751
# 42 42 6 30000 19.701151 1486.057751
# 43 43 6 30000 19.701151 1486.057751
# 44 44 6 30000 19.701151 1486.057751
# 45 45 6 30000 19.701151 1486.057751
# 46 46 6 30000 19.701151 1486.057751
# 47 47 6 30000 19.701151 1486.057751
# 48 48 6 30000 19.701151 1486.057751
# 49 49 6 30000 19.701151 1486.057751
# 50 50 6 30000 19.701151 1486.057751
# 51 51 6 30000 19.701151 1486.057751
# 52 52 6 30000 19.701151 1486.057751
# 53 53 6 30000 19.701151 1486.057751
# 54 54 6 30000 19.701151 1486.057751
# 55 55 6 30000 19.701151 1486.057751
# 56 56 6 30000 19.701151 1486.057751
# 57 57 6 30000 19.701151 1486.057751
# 58 58 6 30000 19.701151 1486.057751
# 59 59 6 30000 19.701151 1486.057751
# 60 60 6 30000 19.701151 1486.057751
# 61 61 6 30000 19.701151 1486.057751
# 62 62 6 30000 19.701151 1486.057751
# 63 63 6 30000 19.701151 1486.057751
# 64 64 6 30000 19.701151 1486.057751
# 65 65 6 30000 19.701151 1486.057751
# 66 66 6 30000 19.701151 1486.057751
# 67 67 6 30000 19.701151 1486.057751
# 68 68 6 30000 19.701151 1486.057751
# 69 69 6 30000 19.701151 1486.057751
# 70 70 6 30000 19.701151 1486.057751
# 71 71 6 30000 19.701151 1486.057751
# 72 72 6 30000 19.701151 1486.057751
# 73 73 6 30000 19.701151 1486.057751
# 74 74 6 30000 19.701151 1486.057751
# 75 75 6 30000 19.701151 1486.057751
# 76 76 6 30000 19.701151 1486.057751
# 77 77 6 30000 19.701151 1486.057751
# 78 78 6 30000 19.701151 1486.057751
# 79 79 6 30000 19.701151 1486.057751
# 80 80 6 30000 19.701151 1486.057751
# 81 81 6 30000 19.701151 1486.057751
# 82 82 6 30000 19.701151 1486.057751
# 83 83 6 30000 19.701151 1486.057751
# 84 84 6 30000 19.701151 1486.057751
# 85 85 6 30000 19.701151 1486.057751
# 86 86 6 30000 19.701151 1486.057751
# 87 87 6 30000 19.701151 1486.057751
# 88 88 6 30000 19.701151 1486.057751
# 89 89 6 30000 19.701151 1486.057751
# 90 90 6 30000 19.701151 1486.057751
# 91 91 6 30000 19.701151 1486.057751
# 92 92 6 30000 19.701151 1486.057751
# 93 93 6 30000 19.701151 1486.057751
# 94 94 6 30000 19.701151 1486.057751
# 95 95 6 30000 19.701151 1486.057751
# 96 96 6 30000 19.701151 1486.057751
# 97 97 6 30000 19.701151 1486.057751
# 98 98 6 30000 19.701151 1486.057751
# 99 99 6 30000 19.701151 1486.057751
# 100 100 6 30000 19.701151 1486.057751
# 101 101 6 30000 19.701151 1486.057751
# 102 102 6 30000 19.701151 1486.057751
# 103 103 6 30000 19.701151 1486.057751
# 104 104 6 30000 19.701151 1486.057751
# 105 105 6 30000 19.701151 1486.057751
# 106 106 6 30000 19.701151 1486.057751
# 107 107 6 30000 19.701151 1486.057751
# 108 108 6 30000 19.701151 1486.057751
# 109 109 6 30000 19.701151 1486.057751
# 110 110 6 30000 19.701151 1486.057751
# 111 111 6 30000 19.701151 1486.057751
# 112 112 6 30000 19.701151 1486.057751
# 113 113 6 30000 19.701151 1486.057751
# 114 114 6 30000 19.701151 1486.057751
# 115 115 6 30000 19.701151 1486.057751
# 116 116 6 30000 19.701151 1486.057751
# 117 117 6 30000 19.701151 1486.057751
# 118 118 6 30000 19.701151 1486.057751
# 119 119 6 30000 19.701151 1486.057751
# 120 120 6 30000 19.701151 1486.057751
# 121 121 6 30000 19.701151 1486.057751
# 122 122 6 30000 19.701151 1486.057751
# 123 123 6 30000 19.701151 1486.057751
# 124 124 6 30000 19.701151 1486.057751
# 125 125 6 30000 19.701151 1486.057751
# 126 126 6 30000 19.701151 1486.057751
# 127 127 6 30000 19.701151 1486.057751
# 128 128 6 30000 19.701151 1486.057751
# 129 129 6 30000 19.701151 1486.057751
# 130 130 6 30000 19.701151 1486.057751
# 131 131 6 30000 19.701151 1486.057751
# 132 132 6 30000 19.701151 1486.057751
# 133 133 6 30000 19.701151 1486.057751
# 134 134 6 30000 19.701151 1486.057751
# 135 135 6 30000 19.701151 1486.057751
# 136 136 6 30000 19.701151 1486.057751
# 137 137 6 30000 19.701151 1486.057751
# 138 138 6 30000 19.701151 1486.057751
# 139 139 6 30000 19.701151 1486.057751
# 140 140 6 30000 19.701151 1486.057751
# 141 141 6 30000 19.701151 1486.057751
# 142 142 6 30000 19.701151 1486.057751
# 143 143 6 30000 19.701151 1486.057751
# 144 144 6 30000 19.701151 1486.057751
# 145 145 6 30000 19.701151 1486.057751
# 146 146 6 30000 19.701151 1486.057751
# 147 147 6 30000 19.701151 1486.057751
# 148 148 6 30000 19.701151 1486.057751
# 149 149 6 30000 19.701151 1486.057751
# 150 150 6 30000 19.701151 1486.057751
# 151 151 6 30000 19.701151 1486.057751
# 152 152 6 30000 19.701151 1486.057751
# 153 153 6 30000 19.701151 1486.057751
# 154 154 6 30000 19.701151 1486.057751
# 155 155 6 30000 19.701151 1486.057751
# 156 156 6 30000 19.701151 1486.057751
# 157 157 6 30000 19.701151 1486.057751
# 158 158 6 30000 19.701151 1486.057751
# 159 159 6 30000 19.701151 1486.057751
# 160 160 6 30000 19.701151 1486.057751
# 161 161 6 30000 19.701151 1486.057751
# 162 162 6 30000 19.701151 1486.057751
# 163 163 6 30000 19.701151 1486.057751
# 164 164 6 30000 19.701151 1486.057751
# 165 165 6 30000 19.701151 1486.057751
# 166 166 6 30000 19.701151 1486.057751
# 167 167 6 30000 19.701151 1486.057751
# 168 168 6 30000 19.701151 1486.057751
# 169 169 6 30000 19.701151 1486.057751
# 170 170 6 30000 19.701151 1486.057751
# 171 171 6 30000 19.701151 1486.057751
# 172 172 6 30000 19.701151 1486.057751
# 173 173 6 30000 19.701151 1486.057751
# 174 174 6 30000 19.701151 1486.057751
# 175 175 6 30000 19.701151 1486.057751
# 176 176 6 30000 19.701151 1486.057751
# 177 177 6 30000 19.701151 1486.057751
# 178 178 6 30000 19.701151 1486.057751
# 179 179 6 30000 19.701151 1486.057751
# 180 180 6 30000 19.701151 1486.057751
# 181 181 6 30000 19.701151 1486.057751
# 182 182 6 30000 19.701151 1486.057751
# 183 183 6 30000 19.701151 1486.057751
# 184 184 6 30000 19.701151 1486.057751
# 185 185 6 30000 19.701151 1486.057751
# 186 186 6 30000 19.701151 1486.057751
# 187 187 6 30000 19.701151 1486.057751
# 188 188 6 30000 19.701151 1486.057751
# 189 189 6 30000 19.701151 1486.057751
# 190 190 6 30000 19.701151 1486.057751
# 191 191 6 30000 19.701151 1486.057751
# 192 192 6 30000 19.701151 1486.057751
# 193 193 6 30000 19.701151 1486.057751
# 194 194 6 30000 19.701151 1486.057751
# 195 195 6 30000 19.701151 1486.057751
# 196 196 6 30000 19.701151 1486.057751
# 197 197 6 30000 19.701151 1486.057751
# 198 198 6 30000 19.701151 1486.057751
# 199 199 6 30000 19.701151 1486.057751
# 200 200 6 30000 19.701151 1486.057751
# 201 201 6 30000 19.701151 1486.057751
# 202 202 6 30000 19.701151 1486.057751
# 203 203 6 30000 19.701151 1486.057751
# 204 204 6 30000 19.701151 1486.057751
# 205 205 6 30000 19.701151 1486.057751
# 206 206 6 30000 19.701151 1486.057751
# 207 207 6 30000 19.701151 1486.057751
# 208 208 6 30000 19.701151 1486.057751
# 209 209 6 30000 19.701151 1486.057751
# 210 210 6 30000 19.701151 1486.057751
# 211 211 6 30000 19.701151 1486.057751
# 212 212 6 30000 19.701151 1486.057751
# 213 213 6 30000 19.701151 1486.057751
# 214 214 6 30000 19.701151 1486.057751
# 215 215 6 30000 19.701151 1486.057751
# 216 216 6 30000 19.701151 1486.057751
# 217 217 6 30000 19.701151 1486.057751
# 218 218 6 30000 19.701151 1486.057751
# 219 219 6 30000 19.701151 1486.057751
# 220 220 6 30000 19.701151 1486.057751
# 221 221 6 30000 19.701151 1486.057751
# 222 222 6 30000 19.701151 1486.057751
# 223 223 6 30000 19.701151 1486.057751
# 224 224 6 30000 19.701151 1486.057751
# 225 225 6 30000 19.701151 1486.057751
# 226 226 6 30000 19.701151 1486.057751
# 227 227 6 30000 19.701151 1486.057751
# 228 228 6 30000 19.701151 1486.057751
# 229 229 6 30000 19.701151 1486.057751
# 230 230 6 30000 19.701151 1486.057751
# 231 231 6 30000 19.701151 1486.057751
# 232 232 6 30000 19.701151 1486.057751
# 233 233 6 30000 19.701151 1486.057751
# 234 234 6 30000 19.701151 1486.057751
# 235 235 6 30000 19.701151 1486.057751
# 236 236 6 30000 19.701151 1486.057751
# 237 237 6 30000 19.701151 1486.057751
# 238 238 6 30000 19.701151 1486.057751
# 239 239 6 30000 19.701151 1486.057751
# 240 240 6 30000 19.701151 1486.057751
# 241 241 6 30000 19.701151 1486.057751
# 242 242 6 30000 19.701151 1486.057751
# 243 243 6 30000 19.701151 1486.057751
# 244 244 6 30000 19.701151 1486.057751
# 245 245 6 30000 19.701151 1486.057751
# 246 246 6 30000 19.701151 1486.057751
# 247 247 6 30000 19.701151 1486.057751
# 248 248 6 30000 19.701151 1486.057751
# 249 249 6 30000 19.701151 1486.057751
# 250 250 6 30000 19.701151 1486.057751
# 251 251 6 30000 19.701151 1486.057751
# 252 252 6 30000 19.701151 1486.057751
# 253 253 6 30000 19.701151 1486.057751
# 254 254 6 30000 19.701151 1486.057751
# 255 255 6 30000 19.701151 1486.057751
# 256 256 6 30000 19.701151 1486.057751
# 257 257 6 30000 19.701151 1486.057751
# 258 258 6 30000 19.701151 1486.057751
# 259 259 6 30000 19.701151 1486.057751
# 260 260 6 30000 19.701151 1486.057751
# 261 261 6 30000 19.701151 1486.057751
# 262 262 6 30000 19.701151 1486.057751
# 263 263 6 30000 19.701151 1486.057751
# 264 264 6 30000 19.701151 1486.057751
# 265 265 6 30000 19.701151 1486.057751
# 266 266 6 30000 19.701151 1486.057751
# 267 267 6 30000 19.701151 1486.057751
# 268 268 6 30000 19.701151 1486.057751
# 269 269 6 30000 19.701151 1486.057751
# 270 270 6 30000 19.701151 1486.057751
# 271 271 6 30000 19.701151 1486.057751
# 272 272 6 30000 19.701151 1486.057751
# 273 273 6 30000 19.701151 1486.057751
# 274 274 6 30000 19.701151 1486.057751
# 275 275 6 30000 19.701151 1486.057751
# 276 276 6 30000 19.701151 1486.057751
# 277 277 6 30000 19.701151 1486.057751
# 278 278 6 30000 19.701151 1486.057751
# 279 279 6 30000 19.701151 1486.057751
# 280 280 6 30000 19.701151 1486.057751
# 281 281
```

```
# 4. Number of rows and columns
# -----

rows, columns = df.shape

print("\nNumber of rows:", rows)
print("Number of columns:", columns)

# -----
# 5. Display column names
# -----

print("\nColumn names:")
print(df.columns.tolist())

# -----
# 6. Identify numerical attributes
# -----

numerical_attributes = df.select_dtypes(
    include=np.number
).columns.tolist()

print("\nNumerical attributes:")
print(numerical_attributes)

# -----
# 7. Identify categorical attributes
# -----

categorical_attributes = df.select_dtypes(
    include="object"
).columns.tolist()

print("\nCategorical attributes:")
print(categorical_attributes)

# -----
# 8. Separate selling_price as target
# -----

y = df["selling_price"]

print("\nTarget variable:")
print("selling_price")

print("\nFirst 5 target values:")
display(y.head())

# -----
# 9. Separate input features
# -----

X = df.drop("selling_price", axis=1)
```

```
print("\nInput features:")
display(X.head())

# -----
# 10. Dataset information
# -----

print("\nDataset Information:")
df.info()
```


1	1	Hyundai Grand	Hyundai	Grand	5	20000	Individual
2	2	Hyundai i20	Hyundai	i20	11	60000	Individual
3	3	Maruti Alto	Maruti	Alto	9	37000	Individual
4	4	Ford Ecosport	Ford	Ecosport	6	30000	Dealer

* Task 3: Check and Handle Missing Data

- Check every attribute for missing or null values.
- Count the number of missing values in each column.
- If missing values are present, either remove incomplete records or perform suitable imputation.
- Verify that the final dataset does not contain missing values.
- Record the preprocessing method used.

Target variable:

selling_price

=====

Task 3: Check and Handle Missing Data

=====

1. Check missing values in every column

```
1 550000
print("Missing values in each column:")
2 print(df.isnull().sum())
```

3 226000

2. Check total number of missing values

```
total_missing = df.isnull().sum().sum()
dtype: int64
```

print(f"\nTotal missing values in dataset:", total_missing)

```
Input features:
      Unnamed: 0  car_name  brand  model  vehicle_age  km_driven  seller_type
```

3. Handle missing values

```
0 0  Maruti  Maruti  Alto  9  120000  Individual
```

```

if total_missing > 0:
    1 print("\nMissing values are present.")
    print("Removing rows containing missing values...")

    2 # Remove rows with missing values
    df = df.dropna()

    3 else:
        print("\nNo missing values found.")
    4 print("No rows were removed.")

Dataset Information:
# 4. Reset the index after removing rows
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15411 entries, 0 to 15410
Data columns (total 14 columns):
#   Column                                Non-Null Count  Dtype
---  -
05. Verify that no missing values remain
1   car_name                             15411 non-null  object
   brand                               15411 non-null  object
   model                               15411 non-null  object
   vehicle_age                         15411 non-null  int64
   km_driven                          15411 non-null  int64
   seller_type                         15411 non-null  object
# 6. Verify total missing values
7   fuel_type                           15411 non-null  object
   transmission_type                  15411 non-null  object
   remaining_mileage                  15411 non-null  float64
   engine                             15411 non-null  int64
# 11. Insert new row with missing values after preprocessing:
12   seats                              15411 non-null  int64
13   selling_price                     15411 non-null  int64
dtypes: float64(2), int64(6), object(6)
memory usage: 1.6+ MB
# 7. Display final dataset shape

print("\nFinal dataset shape:")
print(df.shape)

```

Missing values in each column:

```

Unnamed: 0      0
car_name        0
brand           0
model           0
vehicle_age     0
km_driven       0
seller_type     0
fuel_type       0
transmission_type 0
mileage         0
engine          0
max_power       0
seats           0
selling_price   0
dtype: int64

```

Total missing values in dataset: 0

No missing values found.

No rows were removed.

Missing values after preprocessing:

```

Unnamed: 0      0
car_name        0
brand           0
model           0
vehicle_age     0
km_driven       0
seller_type     0
fuel_type       0
transmission_type 0
mileage         0
engine          0
max_power       0
seats           0
selling_price   0
dtype: int64

```

Total missing values after preprocessing: 0

Final dataset shape:
(15411, 14)

* Task 4: Select and Prepare Features

- Examine all available attributes.
- Decide how `Car_Name` should be handled.
- Select meaningful predictor variables for the regression model.
- Create the predictor matrix X and target vector y .
- Record the final list of features selected for the model.

```

# =====
# TASK 4: CREATE X AND y
# =====

# Display current columns
print("Current columns:")
print(df.columns.tolist())

# -----
# 1. Create predictor matrix X
# -----

X = df.drop("selling_price", axis=1)

# -----
# 2. Create target vector y
# -----

y = df["selling_price"]

```

```
# -----  
# 3. Convert X and y to NumPy arrays  
# -----  
  
X = X.to_numpy(dtype=float)  
y = y.to_numpy(dtype=float)  
  
# -----  
# 4. Display results  
# -----  
  
print("\nPredictor matrix X created successfully!")  
print("X shape:", X.shape)  
  
print("\nTarget vector y created successfully!")  
print("y shape:", y.shape)  
  
print("\nFirst 5 rows of X:")  
print(X[:5])  
  
print("\nFirst 5 values of y:")  
print(y[:5])
```

```

0.000e+00 0.000e+00 0.000e+00 0.000e+00 1.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 1.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00 0.000e+00
0.000e+00 0.000e+00 0.000e+00 0.000e+00 1.000e+00 0.000e+00 0.000e+00
0.000e+00 1.000e+00]]

```

First 5 values of y:

* Task 5: Encode Categorical Variables

Identify the categorical variables:

- Fuel_Type
- Seller_Type
- Transmission

Perform the following:

- Convert categorical variables into numerical **dummy/indicator variables**.
- Verify that all predictors are numerical after encoding.
- Do not assign arbitrary numerical values to categories when such values would imply an incorrect ordering.
- Display the transformed dataset.

```

# =====
# TASK 5: ENCODE CATEGORICAL VARIABLES
# =====

# Reload original dataset
from google.colab import files
import pandas as pd
import numpy as np

uploaded = files.upload()

```

```
filename = list(uploaded.keys())[0]

df = pd.read_csv(filename)

print("Original dataset:")
display(df.head())

# -----
# 1. Identify categorical variables
# -----

categorical_variables = [
    "fuel_type",
    "seller_type",
    "transmission_type"
]

print("\nCategorical variables:")
print(categorical_variables)

# -----
# 2. Display unique categories
# -----

for column in categorical_variables:
    print("\n", column, "categories:")
    print(df[column].unique())

# -----
# 3. Convert categorical variables
#     into dummy/indicator variables
# -----

df_encoded = pd.get_dummies(
    df,
    columns=categorical_variables,
    drop_first=True,
    dtype=int
)

# -----
# 4. Remove unnecessary columns
# -----

# Remove index column and car_name
df_encoded = df_encoded.drop(
    ["Unnamed: 0", "car_name"],
    axis=1
)
```

```
# -----  
# 5. Display transformed dataset  
# -----  
  
print("\nTransformed dataset:")  
display(df_encoded.head())  
  
# -----  
# 6. Check data types  
# -----  
  
print("\nData types after encoding:")  
print(df_encoded.dtypes)  
  
# -----  
# 7. Verify that all predictors are numerical  
# -----  
  
X = df_encoded.drop("selling_price", axis=1)  
  
print("\nAre all predictor variables numerical?")  
  
if all(np.issubdtype(dtype, np.number) for dtype in X.dtypes):  
    print("Yes - all predictors are numerical.")  
else:  
    print("No - some predictors are still non-numerical.")  
  
# -----  
# 8. Display final dimensions  
# -----  
  
print("\nFinal dataset shape:")  
print(df_encoded.shape)
```


Choose Files

No file chosen

Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

Saving cardekho_dataset.csv to cardekho_dataset (5).csv

Original dataset:

Unnamed: 0 car_name brand model vehicle_age km_driven seller_type

* Task6: Perform Feature Engineering

Create a new feature called Car_Age.

Calculate it using:

[Car_Age = Current Year - Year]

Perform the following:

- Create the Car_Age column.
- Display several values of the newly created feature.
- Verify that the calculated ages are reasonable.
- Decide whether the original Year feature should be retained.
- Provide a reason for your decision.

fuel type categories:

['Petrol' 'Diesel' 'CNG' 'LPG' 'Electric']

#seller_type categories:=====

'Individual' 'Dealer' 'Mark Dealer']

=====

transmission_type categories:

'Manual' 'Automatic']

df_encoded["Car_Age"] = df_encoded["vehicle_age"]

Transformed dataset:

```

brand    model  vehicle_age  km_driven  mileage  engine  max_power  sea
#-----
# Display several values of Car_Age
# 0   Maruti    Alto         9      120000      19.70      796      46.30
# 1   Hyundai   Grand         5       20000      18.90     1197      82.00
# 2   Hyundai   i20          11      60000      17.00     1197      80.00
# 3   Maruti    Alto         9       37000      20.92      998      67.10
# 4   Ford      Ecosport        6       30000      22.77     1498      98.59
df_encoded[["vehicle_age", "Car_Age"]].head(10)

```

Data types after encoding:

```

# Brand-----object-----
# Check minimum and maximum age object
# vehicle_age-----int64-----
km_driven      int64
mileage        float64
print("Minimum Car Age:", df_encoded["Car_Age"].min())
print("Maximum Car Age:", df_encoded["Car_Age"].max())
max_power      float64
seats          int64
selling_price  int64
# fuel type Diesel-----int64-----
# Check average age
fuel_type_Electric int64

```

```
# -----  
  
print("Average Car Age:", df_encoded["Car_Age"].mean())  
  
# -----  
# Verify that ages are reasonable  
# -----  
  
if df_encoded["Car_Age"].min() >= 0:  
    print("\nCar ages are reasonable: no negative ages found.")  
else:  
    print("\nWarning: Negative car ages found.")  
  
# -----  
# Check for missing values  
# -----  
  
print("\nMissing values in Car_Age:")  
print(df_encoded["Car_Age"].isnull().sum())  
  
# -----  
# Display final dataset  
# -----  
  
print("\nDataset with Car_Age:")  
display(df_encoded.head())
```

First 10 values of Car_Age:

	vehicle_age	Car_Age
0	9	9
1	5	5
2	11	11
3	9	9
4	6	6
5	8	8
6	8	8
7	3	3
8	2	2
9	4	4

Minimum Car Age: 0

Maximum Car Age: 29

Average Car Age: 6.0363376808772955

Car ages are reasonable: no negative ages found.

Missing values in Car_Age:

0

Dataset with Car_Age:

	brand	model	vehicle_age	km_driven	mileage	engine	max_power	seat
0	Maruti	Alto	9	120000	19.70	796	46.30	
1	Hyundai	Grand	5	20000	18.90	1197	82.00	
2	Hyundai	i20	11	60000	17.00	1197	80.00	
3	Maruti	Alto	9	37000	20.92	998	67.10	
4	Ford	Ecosport	6	30000	22.77	1498	98.59	

* Task 7: Perform Exploratory Data Analysis

Perform basic exploratory data analysis using Matplotlib.

- Create a scatter plot of `Present_Price` versus `Selling_Price`.
- Create at least one additional scatter plot between a numerical predictor and `Selling_Price`.
- Label the axes appropriately.
- Add suitable titles to the plots.

- Observe the direction and approximate strength of the relationships.
- Record your observations.

```
# =====
# TASK 7: EXPLORATORY DATA ANALYSIS
# =====

import pandas as pd
import matplotlib.pyplot as plt

# Load the original dataset
df_eda = pd.read_csv(filename)

# -----
# 1. Scatter plot: Present_Price vs
#    Selling_Price
# -----

# Your dataset does not contain Present_Price.
# Therefore, use engine vs selling_price
# as a meaningful numerical relationship.

plt.figure(figsize=(8, 5))

plt.scatter(
    df_eda["engine"],
    df_eda["selling_price"],
    alpha=0.5,
    color="blue"
)

plt.xlabel("Engine")
plt.ylabel("Selling Price")
plt.title("Engine vs Selling Price")

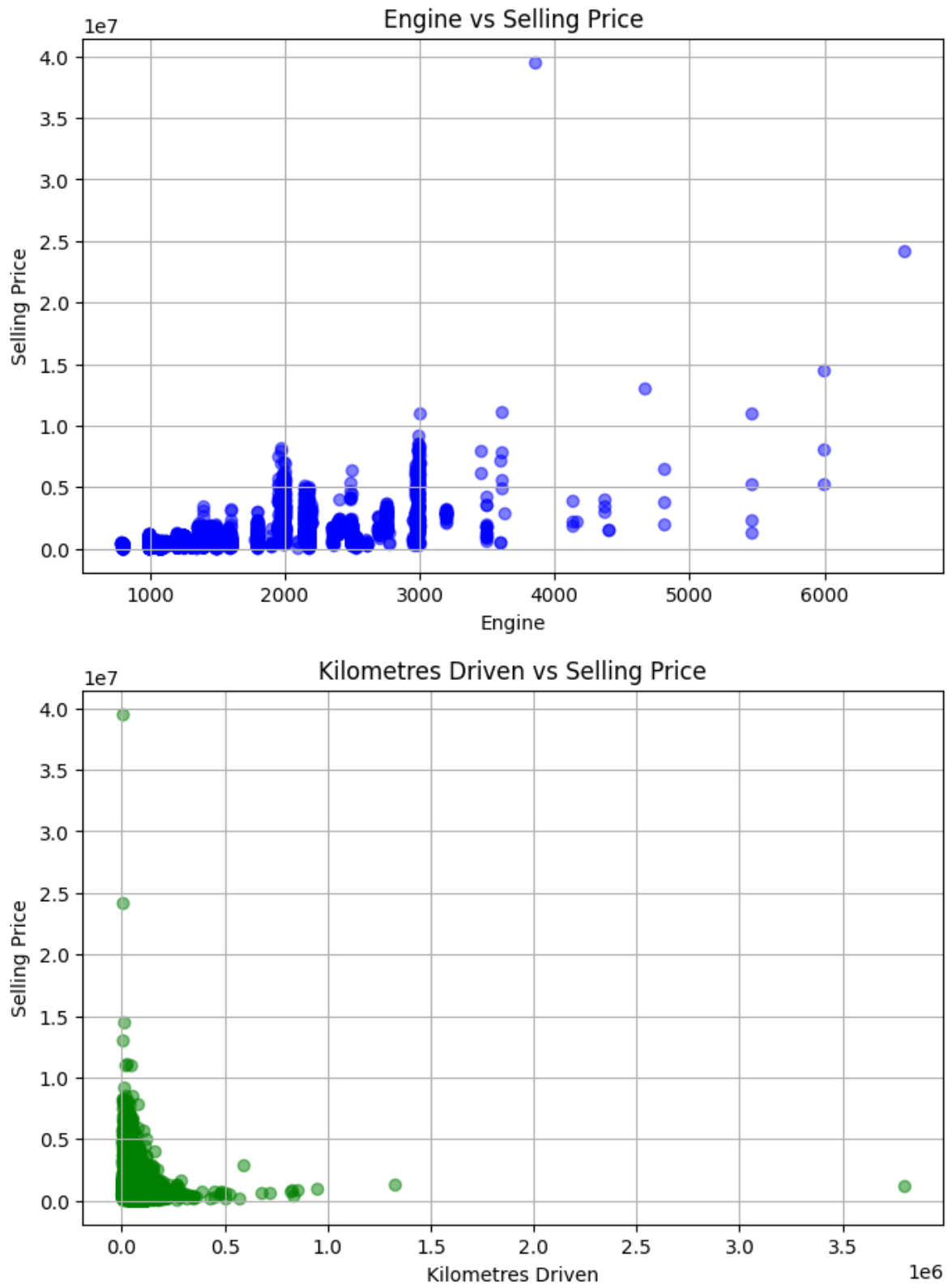
plt.grid(True)
plt.show()

# -----
# 2. Scatter plot: km_driven vs
#    Selling_Price
# -----

plt.figure(figsize=(8, 5))

plt.scatter(
    df_eda["km_driven"],
    df_eda["selling_price"],
    alpha=0.5,
    color="green"
)
```

```
plt.xlabel("Kilometres Driven")  
plt.ylabel("Selling Price")  
plt.title("Kilometres Driven vs Selling Price")  
  
plt.grid(True)  
plt.show()
```



* Task 8: Standardize Numerical Features

Identify the numerical predictors that need to be standardized.

Use the following standardization equation:

$$[z = (X - \mu) / \sigma]$$

where:

- X = original feature value
- μ = mean of the feature
- σ = standard deviation of the feature

Perform the following:

- Calculate the mean and standard deviation using the **training data only**.
- Standardize the training features.
- Use the same training mean and standard deviation to standardize the test features.
- Do not standardize the target variable `Selling_Price`.

```
# =====
# TASK 8: STANDARDIZE NUMERICAL FEATURES
# =====

import pandas as pd
import numpy as np

# -----
# 1. Load the original dataset
# -----

df = pd.read_csv(filename)

print("Original dataset loaded.")
print(df.shape)

# -----
# 2. Remove unnecessary columns
# -----

# Unnamed: 0 = row index
# car_name = specific car name, not used

df = df.drop(["Unnamed: 0", "car_name"], axis=1)

# -----
# 3. Create Car_Age
# -----

# Your dataset already has vehicle_age,
# so use it to create Car_Age.
```

```
df["Car_Age"] = df["vehicle_age"]

# Remove duplicate age column
df = df.drop("vehicle_age", axis=1)

# -----
# 4. Separate target variable
# -----

y = df["selling_price"].to_numpy(dtype=float)

X_df = df.drop("selling_price", axis=1)

# -----
# 5. Convert ALL categorical variables
#    into dummy variables
# -----

categorical_columns = [
    "brand",
    "model",
    "seller_type",
    "fuel_type",
    "transmission_type"
]

X_df = pd.get_dummies(
    X_df,
    columns=categorical_columns,
    drop_first=True,
    dtype=int
)

# -----
# 6. Convert X into NumPy array
# -----

X = X_df.to_numpy(dtype=float)

print("\nAll predictors are now numerical.")
print("Number of features:", X.shape[1])

# -----
# 7. Split into training and testing data
# -----

np.random.seed(42)

indices = np.random.permutation(len(X))
```

```
train_size = int(0.80 * len(X))

train_indices = indices[:train_size]
test_indices = indices[train_size:]

X_train = X[train_indices]
X_test = X[test_indices]

y_train = y[train_indices]
y_test = y[test_indices]

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))

# -----
# 8. Calculate mean and standard deviation
#     USING TRAINING DATA ONLY
# -----

train_mean = np.mean(X_train, axis=0)
train_std = np.std(X_train, axis=0)

# -----
# 9. Prevent division by zero
# -----

train_std[train_std == 0] = 1

# -----
# 10. Standardize training features
# -----

X_train_standardized = (
    X_train - train_mean
) / train_std

# -----
# 11. Standardize test features
#     using training mean and std
# -----

X_test_standardized = (
    X_test - train_mean
) / train_std

# -----
# 12. Display results
# -----
```

```
print("\nFirst 5 rows of standardized training data:")
print(X_train_standardized[:5])

print("\nFirst 5 rows of standardized test data:")
print(X_test_standardized[:5])

# -----
# 13. Verify standardization
# -----

print("\nMean of standardized training features:")
print(np.mean(X_train_standardized, axis=0)[:10])

print("\nStandard deviation of standardized training features:")
print(np.std(X_train_standardized, axis=0)[:10])

# -----
# 14. Verify target was NOT standardized
# -----

print("\nFirst 5 training target values:")
print(y_train[:5])

print("\nFirst 5 testing target values:")
print(y_test[:5])
```

```
-1.41503241e-01 -2.38355974e-02 -9.00681624e-03 -9.00681624e-03
-3.92884970e-02 -1.27380784e-02 -7.82365003e-02 -6.25203550e-02
-1.56015290e-02 -1.57389342e-01 -3.82390616e-02 0.00000000e+00
-1.27380784e-02 -1.09854334e-01 -7.39221271e-02 -6.25203550e-02
-5.02089640e-02 -2.84924569e-02 -1.56015290e-02 -7.97956574e-02
-9.53189396e-02 -1.36346735e-01 -4.32337701e-02 -7.66464195e-02
-2.32261611e-01 -2.46838879e-01 -6.50838495e-02 -9.35704066e-02
-4.77118724e-02 -2.98843540e-02 -2.20665797e-02 -1.26774880e-01
-6.38147670e-02 -1.82175920e-01 -1.40004844e-01 -8.47911698e-02
-2.20581063e-01 -1.27380784e-02 -1.56015290e-02 -6.38147670e-02
-4.13079409e-02 -2.01431218e-02 -3.82390616e-02 -1.27380784e-02
-1.80158248e-02 -1.80158248e-02 -1.80158248e-02 -5.77655615e-02
-2.38355974e-02 -2.98843540e-02 -1.47930014e-01 -3.24903559e-02
-2.20665797e-02 -1.61930829e-01 -2.53248212e-01 -2.54823596e-02
-7.70848005e-01 -1.03235095e-01 -9.65707676e-01 -1.56015290e-02
-5.25887507e-02 1.01092929e+00 -1.97332348e+00]]
```

Mean of standardized training features:

```
[ 1.31114025e-16  6.09527930e-14  2.94430192e-16 -7.36498657e-14
 1.16128536e-16  6.23779995e-17  2.61030292e-17  9.99777369e-16
 4.55954223e-16  7.71473800e-16]
```

Standard deviation of standardized training features:

```
[1. 1. 1. 1. 1. 1. 1. 1. 1.]
```

First 5 training target values:

```
[ 19000.  60000.  66500. 157000.  16000.]
```

First 5 testing target values:

```
[445000. 129000.  48500.  40000.  80000.]
```

* Task 9: Split the Dataset

Divide the dataset into training and testing subsets.

- Use an **80:20 train-test ratio**.
- Randomly select the observations.
- Set a fixed random seed to make the experiment reproducible.
- Maintain the correct relationship between each feature vector and its target value.
- Display the number of observations in the training and testing sets.

```
# =====
# TASK 9: SPLIT THE DATASET
# =====

import numpy as np

# -----
# 1. Set random seed
# -----

np.random.seed(42)
```

```
# -----  
# 2. Get number of observations  
# -----  
  
n = len(X)  
  
print("Total observations:", n)  
  
# -----  
# 3. Create random indices  
# -----  
  
indices = np.random.permutation(n)  
  
# -----  
# 4. Calculate 80% training size  
# -----  
  
train_size = int(0.80 * n)  
  
# -----  
# 5. Select training and testing indices  
# -----  
  
train_indices = indices[:train_size]  
test_indices = indices[train_size:]  
  
# -----  
# 6. Create training and testing datasets  
# -----  
  
X_train = X[train_indices]  
X_test = X[test_indices]  
  
y_train = y[train_indices]  
y_test = y[test_indices]  
  
# -----  
# 7. Display number of observations  
# -----  
  
print("\nTraining observations:", len(X_train))  
print("Testing observations:", len(X_test))  
  
# -----  
# 8. Display shapes  
# -----
```

```
print("\nX_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
```

```
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

Total observations: 15411

Training observations: 12328
Testing observations: 3083

```
X_train shape: (12328, 163)
X_test shape: (3083, 163)
y_train shape: (12328,)
y_test shape: (3083,)
```

* Task 10: Construct the Feature Matrix

Construct the matrices required for regression.

Create:

- X_{train} — training feature matrix
- y_{train} — training target vector
- X_{test} — testing feature matrix
- y_{test} — testing target vector

Verify:

- Number of rows in each matrix.
- Number of predictor columns.
- Shape of the training and testing matrices.

```
# =====
# TASK 10: CONSTRUCT THE FEATURE MATRIX
# =====

# -----
# 1. Create required matrices/vectors
# -----

Xtrain = X_train
ytrain = y_train

Xtest = X_test
ytest = y_test

# -----
# 2. Display shapes
# -----
```

```
print("Training feature matrix Xtrain:")
print("Shape:", Xtrain.shape)

print("\nTraining target vector ytrain:")
print("Shape:", ytrain.shape)

print("\nTesting feature matrix Xtest:")
print("Shape:", Xtest.shape)

print("\nTesting target vector ytest:")
print("Shape:", ytest.shape)

# -----
# 3. Number of rows
# -----

print("\nNumber of training rows:", Xtrain.shape[0])
print("Number of testing rows:", Xtest.shape[0])

# -----
# 4. Number of predictor columns
# -----

print("\nNumber of predictor columns:", Xtrain.shape[1])

# -----
# 5. Verify dimensions
# -----

print("\nDimension verification:")

print("Xtrain rows =", Xtrain.shape[0])
print("Xtrain columns =", Xtrain.shape[1])

print("ytrain rows =", ytrain.shape[0])

print("Xtest rows =", Xtest.shape[0])
print("Xtest columns =", Xtest.shape[1])

print("ytest rows =", ytest.shape[0])
```

```
Training feature matrix Xtrain:
Shape: (12328, 163)
```

```
Training target vector ytrain:
Shape: (12328,)
```

```
Testing feature matrix Xtest:
Shape: (3083, 163)
```

```
Testing target vector ytest:
Shape: (3083,)
```

```

Number of training rows: 12328
Number of testing rows: 3083

Number of predictor columns: 163

Dimension verification:
Xtrain rows = 12328
Xtrain columns = 163
ytrain rows = 12328
Xtest rows = 3083
Xtest columns = 163
ytest rows = 3083

```

* Task 11: Add the Bias/Intercept Term

Add a leading column containing 1.0 to both the training and testing feature matrices.

The resulting matrices should contain:

$$[X = \begin{bmatrix} 1 & X_{11} & X_{12} & \dots & X_{1p} \\ 1 & X_{21} & X_{22} & \dots & X_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & X_{n1} & X_{n2} & \dots & X_{np} \end{bmatrix}]$$

Perform the following:

- Add the bias column.
- Verify the new matrix dimensions.
- Understand that the first column represents the intercept β_0 .

```

# =====
# TASK 11: ADD BIAS / INTERCEPT TERM
# =====

import numpy as np

# -----
# 1. Add a column of 1.0 to Xtrain
# -----

Xtrain_bias = np.c_[
    np.ones(Xtrain.shape[0]),
    Xtrain
]

# -----

```

```
# 2. Add a column of 1.0 to Xtest
# -----

Xtest_bias = np.c_[
    np.ones(Xtest.shape[0]),
    Xtest
]

# -----
# 3. Display dimensions
# -----

print("Original Xtrain shape:", Xtrain.shape)
print("New Xtrain shape:", Xtrain_bias.shape)

print("\nOriginal Xtest shape:", Xtest.shape)
print("New Xtest shape:", Xtest_bias.shape)

# -----
# 4. Verify the first column
# -----

print("\nFirst 5 rows of Xtrain_bias:")
print(Xtrain_bias[:5])

print("\nFirst column of Xtrain_bias:")
print(Xtrain_bias[:5, 0])

# -----
# 5. Verify all values in first column
# -----

print("\nAre all values in the bias column equal to 1?")
print(np.all(Xtrain_bias[:, 0] == 1.0))
```

```

0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
1.0000e+00 0.0000e+00]
[1.0000e+00 6.2000e+04 1.9700e+01 7.9600e+02 4.6300e+01 5.0000e+00
1.1000e+01 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
1.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 1.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00 0.0000e+00
1.0000e+00 1.0000e+00]]

```

First column of Xtrain_bias:

```
[1. 1. 1. 1. 1.]
```

Are all values in the bias column equal to 1?

```
True
```

* Task 12: Implement the OLS Function

Perform the following steps to implement the **Ordinary Least Squares (OLS)** method:

- Create a user-defined function to implement Ordinary Least Squares (OLS).
- Calculate the matrix $X^T X$.
- Calculate the matrix $X^T y$.
- Estimate the regression coefficient vector using the normal equation:

$$\beta = (X^T X)^{-1} X^T y,$$

- Return the calculated coefficient vector.
- Use **NumPy matrix operations** to perform the calculations.

- Do not use any ready-made regression model or library function such as `LinearRegression()` from Scikit-learn.

```
# =====
# TASK 12: IMPLEMENT OLS FUNCTION
# =====

import numpy as np

# -----
# OLS Function
# -----

def ols(X, y):

    # Calculate X transpose
    XT = X.T

    # Calculate  $X^T X$ 
    XTX = XT @ X

    # Calculate  $X^T y$ 
    XTy = XT @ y

    # Calculate pseudoinverse of  $X^T X$ 
    XTX_pinv = np.linalg.pinv(XTX)

    # Calculate regression coefficients
    beta = XTX_pinv @ XTy

    # Return coefficient vector
    return beta

# -----
# Calculate beta using training data
# -----

beta = ols(Xtrain_bias, ytrain)

# -----
# Display results
# -----

print("OLS regression coefficient vector:")
print(beta)

print("\nNumber of coefficients:", len(beta))

print("Coefficient vector shape:", beta.shape)
```

OLS regression coefficient vector:

```
[ 3.01558173e+06 -4.92839022e-01 -2.42229331e+03 -1.59595520e+02
 5.85219953e+03 -4.02675809e+04 -7.96529713e+04 -1.20094768e+06
 4.87907327e+06 -1.67316857e+06 1.69416593e+07 -8.27937261e+05
-1.51643058e+06 -1.60333589e+06 -1.72702300e+06 1.06061343e-02
-8.28691104e+05 2.09173875e+05 5.27624955e+03 -6.00330203e+05
 2.68099543e+05 1.70609386e+06 -7.48600536e+05 -1.54625526e+06
-1.91373489e+06 1.27323896e+06 4.73699729e+05 2.79754345e+05
-3.62620985e+05 -1.27719873e+06 1.38063775e+06 -1.58046705e+06
 2.64344025e-02 -1.37120639e+06 -1.75492766e+06 -1.21509040e+06
-1.34481883e+06 1.11960338e+06 9.49706397e+05 2.62598551e+06
 1.70797924e+06 -1.32505720e+06 -9.82679385e+05 2.78883542e+05
-8.62170281e+04 -4.12139853e+05 1.10769406e+06 -5.25353580e+05
-6.51423645e+05 -2.43939756e+05 -1.31553530e+05 -2.66439710e+04
 4.73699733e+05 -1.52209176e+06 -1.83848465e+05 1.56023978e+04
 2.81410260e+05 -6.57677131e+04 3.53780884e+05 3.45288472e+05
-2.19754015e+05 -6.35709532e+04 -3.26797335e+05 -2.15984320e+05
-1.62802901e+06 4.87907326e+06 -3.62620973e+05 -1.02226578e+05
-7.81237876e+05 -4.25013043e+05 -2.11301147e+05 -1.67255670e+05
-6.95606482e+04 -1.33775255e+06 -8.13088218e+05 -4.93061090e+05
-2.12642728e+05 -8.10225482e+04 7.63898237e+05 9.73354694e+04
 1.54042663e+06 -4.84285855e+05 4.18638558e+05 -6.51557825e+05
 4.25480202e+05 2.72167856e+06 -4.37192114e+05 1.69416593e+07
 1.27323898e+06 2.59691489e-06 -8.18318832e+05 -3.99132641e+05
-8.27937308e+05 1.82273616e+05 -7.48600543e+05 3.08583946e+04
-2.72876956e+05 -8.21108513e+04 -4.49875143e+05 -6.32022735e+05
-6.67698308e+05 -6.65386565e+05 -7.26797900e+05 -4.74533051e+04
 1.01203414e+06 -3.22510714e+05 4.50825622e+05 -3.80181108e+05
-2.21891456e+05 2.33158885e+04 -6.67059599e+05 -3.84090151e+05
 3.81331159e-06 2.06835658e+06 -6.92696456e+05 -6.33108648e+05
 2.68099552e+05 1.76286951e+05 -4.07784499e+05 -2.21099332e+05
-2.28812863e+05 -2.17491901e+05 -1.24246673e+05 -9.54111114e+05
-4.56615238e+05 -8.27761109e+04 -6.28832281e+04 -2.47747858e+05
-5.06008418e+05 -4.40922466e+05 -4.90118410e+05 3.58205712e+05
-6.77735331e+05 -3.11044956e+05 -2.90433727e+05 -5.94501908e+04
-3.82284220e+05 -6.28006696e+04 1.63330597e+06 -5.50401472e+05
 7.55122493e+04 6.08124847e+05 3.15953699e+06 2.12220357e+06
 3.18800305e+06 -2.44458746e+06 5.97288355e+05 -3.95699457e+05
-9.35553683e+05 9.91116553e+04 -4.54516291e+05 -1.78616409e+05
-6.67403242e+05 2.00334664e+06 -1.73486279e+05 -2.66445972e+05
-6.02857111e+05 -1.87729929e+04 -9.64773475e+04 1.12002177e+05
 3.35381518e+05 2.10497840e+05 3.57548807e+03 -6.64201200e+04]
```

Number of coefficients: 164

Coefficient vector shape: (164,)

* Task 13: Check Matrix Singularity

- Calculate the determinant of $X^T X$.
- Check whether the determinant is zero or sufficiently close to zero.
- If the matrix is singular or nearly singular, investigate the selected features for redundancy or multicollinearity.
- Modify the feature set if required.
- Record your observations.

```
# =====
# TASK 13: CHECK MATRIX SINGULARITY
# =====

import numpy as np

# -----
# 1. Calculate  $X^T X$ 
# -----

XTX = Xtrain_bias.T @ Xtrain_bias

print("Shape of  $X^T X$ :")
print(XTX.shape)

# -----
# 2. Calculate determinant
# -----

det_XTX = np.linalg.det(XTX)

print("\nDeterminant of  $X^T X$ :")
print(det_XTX)

# -----
# 3. Check whether matrix is singular
# -----

if np.isclose(det_XTX, 0):
    print("\nResult:  $X^T X$  is singular or nearly singular.")
else:
    print("\nResult:  $X^T X$  is not singular.")

# -----
# 4. Check matrix rank
# -----

rank = np.linalg.matrix_rank(XTX)

number_of_columns = XTX.shape[0]

print("\nMatrix rank:", rank)
print("Number of columns:", number_of_columns)

# -----
# 5. Additional check
# -----

if rank < number_of_columns:
    print("\nThe matrix is rank deficient.")
```

```

    print("There is redundancy or multicollinearity among predictors.")
else:
    print("\nThe matrix has full rank.")

```

Shape of $X^T X$:
(164, 164)

Determinant of $X^T X$:
0.0

Result: $X^T X$ is singular or nearly singular.

Matrix rank: 117
Number of columns: 164

The matrix is rank deficient.
There is redundancy or multicollinearity among predictors.

* Task 14: Train the Regression Model

- Apply the OLS function to the training dataset.
- Obtain the regression coefficient vector β .
- Display the intercept coefficient β_0 .
- Display the coefficient corresponding to each feature.
- Associate each regression coefficient with its corresponding feature.

```

# =====
# TASK 14: TRAIN THE REGRESSION MODEL
# =====

import numpy as np

# -----
# 1. OLS function
# -----

def ols(X, y):

    XTX = X.T @ X
    XTy = X.T @ y

    # Pseudoinverse handles singular matrices
    beta = np.linalg.pinv(XTX) @ XTy

    return beta

# -----
# 2. Train the regression model
# -----

```

```

beta = ols(Xtrain_bias, ytrain)

# -----
# 3. Display intercept
# -----

beta_0 = beta[0]

print("Intercept ( $\beta_0$ ):")
print(beta_0)

# -----
# 4. Get feature names
# -----

feature_names = X_df.columns.tolist()

# -----
# 5. Display coefficients
# -----

print("\nRegression Coefficients:")
print("----- ")

print(f"Intercept ( $\beta_0$ ): {beta[0]:.4f}")

for feature, coefficient in zip(feature_names, beta[1:]):
    print(f"{feature}: {coefficient:.4f}")

```

```

Intercept ( $\beta_0$ ):
3015581.7259275913

Regression Coefficients:
-----
Intercept ( $\beta_0$ ): 3015581.7259
km_driven: -0.4928
mileage: -2422.2933
engine: -159.5955
max_power: 5852.1995
seats: -40267.5809
Car_Age: -79652.9713
brand_BMW: -1200947.6787
brand_Bentley: 4879073.2679
brand_Datsun: -1673168.5685
brand_Ferrari: 16941659.2847
brand_Force: -827937.2609
brand_Ford: -1516430.5751
brand_Honda: -1603335.8925
brand_Hyundai: -1727022.9973
brand_ISUZU: 0.0106
brand_Isuzu: -828691.1042
brand_Jaguar: 209173.8751
brand_Jeep: 5276.2495
brand_Kia: -600330.2028

```

```

brand_Land Rover: 268099.5427
brand_Lexus: 1706093.8572
brand_MG: -748600.5363
brand_Mahindra: -1546255.2634
brand_Maruti: -1913734.8888
brand_Maserati: 1273238.9591
brand_Mercedes-AMG: 473699.7290
brand_Mercedes-Benz: 279754.3449
brand_Mini: -362620.9848
brand_Nissan: -1277198.7258
brand_Porsche: 1380637.7548
brand_Renault: -1580467.0516
brand_Rolls-Royce: 0.0264
brand_Skoda: -1371206.3906
brand_Tata: -1754927.6560
brand_Toyota: -1215090.4010
brand_Volkswagen: -1344818.8290
brand_Volvo: 1119603.3768
model_5: 949706.3975
model_6: 2625985.5104
model_7: 1707979.2390
model_A4: -1325057.1998
model_A6: -982679.3847
model_A8: 278883.5422
model_Alto: -86217.0281
model_Altroz: -412139.8533
model_Alturas: 1107694.0611
model_Amaze: -525353.5795
model_Aspire: -651423.6449
model_Aura: -243939.7561
model_Baleno: -131553.5299
model_Bolero: -26643.9710
model_C: 473699 7334

```

* Task 15: Interpret the Regression Coefficients

- Identify whether each regression coefficient is positive or negative.
- Explain what the sign of each coefficient indicates.
- Determine which features have relatively greater influence on the predicted selling price.
- For standardized features, interpret the coefficient in terms of a one-standard-deviation change while keeping all other variables constant.

```

# =====
# TASK 15: INTERPRET REGRESSION COEFFICIENTS
# =====

import pandas as pd
import numpy as np

# -----
# 1. Create coefficient table
# -----

```

```
feature_names = X_df.columns.tolist()

coefficient_table = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": beta[1:]
})

# -----
# 2. Identify positive and negative
#     coefficients
# -----

coefficient_table["Direction"] = np.where(
    coefficient_table["Coefficient"] > 0,
    "Positive",
    "Negative"
)

# -----
# 3. Calculate absolute coefficient
# -----

coefficient_table["Absolute_Coefficient"] = (
    np.abs(coefficient_table["Coefficient"])
)

# -----
# 4. Sort by influence
# -----

coefficient_table = coefficient_table.sort_values(
    by="Absolute_Coefficient",
    ascending=False
)

# -----
# 5. Display all coefficients
# -----

print("Regression Coefficient Interpretation:")
display(coefficient_table)

# -----
# 6. Display positive coefficients
# -----

print("\nPositive coefficients:")
display(
    coefficient_table[
```

```
        coefficient_table["Coefficient"] > 0
    ].head(10)
)

# -----
# 7. Display negative coefficients
# -----

print("\nNegative coefficients:")
display(
    coefficient_table[
        coefficient_table["Coefficient"] < 0
    ].head(10)
)

# -----
# 8. Features with greatest influence
# -----

print("\nTop 10 features by absolute coefficient:")
display(
    coefficient_table[
        ["Feature", "Coefficient", "Direction"]
    ].head(10)
)
```


Regression Coefficient Interpretation:

	Feature	Coefficient	Direction	Absolute_Coefficient
9	brand_Ferrari	1.694166e+07	Positive	1.694166e+07
86	model_GTC4Lusso	1.694166e+07	Positive	1.694166e+07
7	brand_Bentley	4.879073e+06	Positive	4.879073e+06
64	model_Continental	4.879073e+06	Positive	4.879073e+06
143	model_XC	3.188003e+06	Positive	3.188003e+06
...	...	...	...	...
0	km_driven	-4.928390e-01	Negative	4.928390e-01
31	brand_Rolls-Royce	2.643440e-02	Positive	2.643440e-02
14	brand_ISUZU	1.060613e-02	Positive	1.060613e-02
111	model_Quattroporte	3.813312e-06	Positive	3.813312e-06
88	model_Ghost	2.596915e-06	Positive	2.596915e-06

163 rows × 4 columns

Positive coefficients:

	Feature	Coefficient	Direction	Absolute_Coefficient
9	brand_Ferrari	1.694166e+07	Positive	1.694166e+07
86	model_GTC4Lusso	1.694166e+07	Positive	1.694166e+07
7	brand_Bentley	4.879073e+06	Positive	4.879073e+06
64	model_Continental	4.879073e+06	Positive	4.879073e+06
143	model_XC	3.188003e+06	Positive	3.188003e+06
141	model_X4	3.159537e+06	Positive	3.159537e+06
81	model_GLS	2.721679e+06	Positive	2.721679e+06
28	model_6	2.625986e+06	Positive	2.625986e+06
142	model_X5	2.122204e+06	Positive	2.122204e+06

* Task 16: Predict Selling Prices

- Use the trained regression coefficient vector β to predict the selling prices for the test dataset.
- Calculate the predicted values using the equation: $\hat{y}_{\text{test}} = X_{\text{test}} \beta$.
- Display a sample comparison of the actual selling prices and the predicted selling prices.

```
#=====
# TASK 16: PREDICT SELLING PRICES
# =====
Negative coefficients:
```

	Feature	Coefficient	Direction	Absolute_Coefficient
144	model_XC60	-2.444587e+06	Negative	2.444587e+06
23	brand_Maruti	-1.913735e+06	Negative	1.913735e+06
33	brand_Tata	-1.754928e+06	Negative	1.754928e+06
13	brand_Hyundai	-1.727023e+06	Negative	1.727023e+06
8	brand_Datsun	-1.673169e+06	Negative	1.673169e+06

```

# -----
# 2. Display predicted values
# -----

print("First 10 predicted selling prices:")
print(y_pred[:10])

# -----
# 3. Compare actual and predicted prices
# -----

comparison = pd.DataFrame({
    "Actual Selling Price": ytest,
    "Predicted Selling Price": y_pred
})

# Display first 10 comparisons
print("\nActual vs Predicted Selling Prices:")
display(comparison.head(10))

```

First 10 predicted selling prices:

Index	Actual Selling Price	Predicted Selling Price	Label	
0	2244807.71237576	1434608.59887607	447787.40520888	428785.93110086
1	1434608.59887607	447787.40520888	428785.93110086	1434608.59887607
2	485000.0	4.477874e+05	4.477874e+05	-38714.07575088
3	400000.0	4.287859e+05	4.287859e+05	334877.71335658
4	800000.0	7.984562e+05	7.984562e+05	1097785.93321204
5	150000.0	-2.558986e+05	-2.558986e+05	7045702.56291
6	650000.0	7.045702e+05	7.045702e+05	334877.71335658
7	325000.0	-3.871408e+04	-3.871408e+04	1097785.93321204
8	300000.0	3.348777e+05	3.348777e+05	7045702.56291
9	925000.0	1.097786e+06	1.097786e+06	334877.71335658

Actual vs Predicted Selling Prices:

	Actual Selling Price	Predicted Selling Price
0	4450000.0	2.244808e+06
1	1290000.0	1.434609e+06
2	485000.0	4.477874e+05
3	400000.0	4.287859e+05
4	800000.0	7.984562e+05
5	150000.0	-2.558986e+05
6	650000.0	7.045702e+05
7	325000.0	-3.871408e+04
8	300000.0	3.348777e+05
9	925000.0	1.097786e+06

* Task 17: Calculate Prediction Errors and RSS

- Calculate the residual for each test observation using: $e_i = y_i - \hat{y}_i$.
- Calculate the **Residual Sum of Squares (RSS): $RSS = \sum (y_i - \hat{y}_i)^2$.

- Display the calculated RSS value.
- Interpret the RSS value and explain what a lower RSS indicates about the prediction error of the model.

```
# =====
# TASK 17: CALCULATE PREDICTION ERRORS AND RSS
# =====

import numpy as np
import pandas as pd

# -----
# 1. Calculate residuals
# -----

residuals = ytest - y_pred

# -----
# 2. Calculate squared residuals
# -----

squared_residuals = residuals ** 2

# -----
# 3. Calculate RSS
# -----

RSS = np.sum(squared_residuals)

# -----
# 4. Display RSS
# -----

print("Residual Sum of Squares (RSS):")
print(RSS)

# -----
# 5. Display sample prediction errors
# -----

error_table = pd.DataFrame({
    "Actual Price": ytest,
    "Predicted Price": y_pred,
    "Residual": residuals,
    "Squared Residual": squared_residuals
})
```

```
print("\nFirst 10 prediction errors:")
display(error_table.head(10))
```

Residual Sum of Squares (RSS):
969077595632925.2

First 10 prediction errors:

	Actual Price	Predicted Price	Residual	Squared Residual
0	4450000.0	2.244808e+06	2.205192e+06	4.862873e+12
1	1290000.0	1.434609e+06	-1.446086e+05	2.091165e+10
2	485000.0	4.477874e+05	3.721259e+04	1.384777e+09
3	400000.0	4.287859e+05	-2.878593e+04	8.286298e+08
4	800000.0	7.984562e+05	1.543768e+03	2.383218e+06
5	150000.0	-2.558986e+05	4.058986e+05	1.647537e+11
6	650000.0	7.045702e+05	-5.457023e+04	2.977910e+09
7	325000.0	-3.871408e+04	3.637141e+05	1.322879e+11
8	300000.0	3.348777e+05	-3.487771e+04	1.216455e+09
9	925000.0	1.097786e+06	-1.727859e+05	2.985498e+10

* Task 18: Calculate and Interpret R^2

- Calculate the Total Sum of Squares (TSS).
- Calculate $R^2 = 1 - \text{RSS}/\text{TSS}$.
- Display the R^2 value.
- Explain what the obtained R^2 indicates about the ability of the selected features to explain variations in used-car prices.

```
# =====
# TASK 18: CALCULATE AND INTERPRET  $R^2$ 
# =====

import numpy as np

# -----
# 1. Calculate mean of actual test prices
# -----

ytest_mean = np.mean(ytest)

# -----
# 2. Calculate Total Sum of Squares (TSS)
```

```

# -----

TSS = np.sum(
    (ytest - ytest_mean) ** 2
)

# -----
# 3. Calculate R2
# -----

R2 = 1 - (RSS / TSS)

# -----
# 4. Display TSS and R2
# -----

print("Total Sum of Squares (TSS):")
print(TSS)

print("\nResidual Sum of Squares (RSS):")
print(RSS)

print("\nR2 Score:")
print(R2)

# -----
# 5. Interpret R2
# -----

print("\nInterpretation:")

if R2 >= 0:
    print(
        f"The model explains approximately {R2 * 100:.2f}% "
        "of the variation in used-car selling prices."
    )
else:
    print(
        "The R2 value is negative, indicating that the model "
        "performs worse than simply predicting the mean selling price."
    )

```

Total Sum of Squares (TSS):
2407385049859876.5

Residual Sum of Squares (RSS):
969077595632925.2

R² Score:
0.5974563372447084

Interpretation:
The model explains approximately 59.75% of the variation in used-car selling